

```
=====
What is Streamlit?
=====
```

**** Streamlit is a Python framework used to create web applications very easily.**

**** In Gen AI projects, Streamlit is commonly used to create simple UIs for:**

- 1) AI Chatbot
- 2) Code Generator
- 3) Text Summarizer
- 4) Resume Analyzer
- 5) Image Generation

**** Streamlit is very useful for Python developers because we can create a UI without learning HTML, CSS, or JavaScript.**

```
=====
Why Streamlit
=====
```

**** Streamlit is useful because:**

- 1) Easy to learn
- 2) Less Code required
- 3) Good for AI/ML/GEN AI project demos
- 4) No frontend knowledge required
- 5) Automatic refresh when UI code changes
- 6) Good for classroom demos and for mini projects

```
=====
Install Streamlit
=====
```

=> We need to install this using pip command

```
$ pip install streamlit
```

```
=====
create a file with app.py
=====
```

```
import streamlit as st
```

```
st.title("Welcome to Streamlit")
st.write("This is my First Streamlit UI Application")
```

```

st.header("Course Details")
st.subheader("Streamlit UI")
st.write("Streamlit is used to create UI using Python.")
st.markdown("### This is markdown text")

name = st.text_input("Enter Student Name")
email = st.text_input("Enter Student Email")
course = st.selectbox(
    "Select Course",
    ["JAVA", "Python", "DevOps", "AI & ML"]
)
uploaded_file = st.file_uploader("Upload Your Resume :", type=["pdf", "docx"])

if st.button("Submit"):
    st.write("Given Name: ", name)
    st.write("Given Email: ", email)
    st.write("Selected Course: ", course)

if uploaded_file is not None:
    st.success("File uploaded successfully")
    st.write("File Name:", uploaded_file.name)

```

Run the streamlit application

```
$ streamlit run app.py
```

```

=====
Important streamlit functions
=====

```

- 1) st.title () : Display page title
- 2) st.write () : Displays text/data
- 3) st.text_input () : Takes single line input using textbox
- 4) st.text_area () : Takes multi-line input
- 5) st.button () : Creates button
- 6) st.selectbox () : Creates dropdown
- 7) st.file_uploader () : Upload files
- 8) st.chat_input () : create chatbot input box
- 9) st.spinner () : shows loading message

```
-----student-api-consumer.py-----
```

```

import streamlit as st
import requests
import pandas as pd

# =====
# FastAPI URL
# =====

```

```
API_URL = "http://127.0.0.1:8000/api"

# =====
# Page Config
# =====
st.set_page_config(
    page_title="Student Management System",
    page_icon="ðŸŽ“",
    layout="wide"
)

# =====
# Header
# =====
st.title("ðŸŽ“ Student Management System")
st.write("### FastAPI + Streamlit REST API Demo")

# =====
# Sidebar
# =====
menu = st.sidebar.selectbox(
    "Select Operation",
    [
        "Create Student",
        "View Students",
        "Search Student"
    ]
)

# =====
# CREATE STUDENT
# =====
if menu == "Create Student":

    st.subheader("â€• Create Student")

    name = st.text_input("Student Name")

    course = st.selectbox(
        "Course",
        [
            "PYTHON",
            "JAVA",
            "DEVOPS",
            "GEN AI",
            "DATA SCIENCE"
        ]
    )

    fee = st.number_input(
        "Course Fee",
        min_value=1.0,
        step=100.0
    )

    if st.button("Save Student"):

        payload = {
            "name": name,
            "course": course,
            "fee": fee
        }
```

```
response = requests.post(
    f"{API_URL}/student",
    json=payload
)

result = response.json()

if response.status_code == 201:

    st.success(result.get("Message", "Student Created Successfully"))

    if "data" in result:

        df = pd.DataFrame([result["data"]])

        st.dataframe(
            df,
            use_container_width=True,
            hide_index=True
        )

    else:
        st.error(result)

# =====
# VIEW STUDENTS
# =====
elif menu == "View Students":

    st.subheader("ðŸ“‹ Student List")

    if st.button("Load Students"):

        response = requests.get(f"{API_URL}/students")

        result = response.json()

        if response.status_code == 200:

            students = result["data"]

            df = pd.DataFrame(students)

            col1, col2, col3 = st.columns(3)

            col1.metric("Total Students", len(df))

            if "FEE" in df.columns:
                col2.metric("Total Fee", f"â‚¹ {df['FEE'].sum():,.0f}")
                col3.metric("Average Fee", f"â‚¹ {df['FEE'].mean():,.0f}")

            st.divider()

            st.dataframe(
                df,
                use_container_width=True,
                hide_index=True
            )

        else:
            st.error(result.get("Message", "Unable to fetch students"))
```

```
# =====
# SEARCH STUDENT
# =====
elif menu == "Search Student":

    st.subheader("ðŸ”” Search Student")

    student_id = st.number_input(
        "Student ID",
        min_value=1,
        step=1
    )

    if st.button("Search"):

        response = requests.get(
            f"{API_URL}/students/{student_id}"
        )

        result = response.json()

        if response.status_code == 200:

            student = result["data"]

            # Handle both object and list responses
            if isinstance(student, dict):
                df = pd.DataFrame([student])
            else:
                df = pd.DataFrame(student)

            st.success(result.get("Message", "Student Found"))

            st.dataframe(
                df,
                use_container_width=True,
                hide_index=True
            )

        else:
            st.error(result.get("Message", "Student Not Found"))
```

<https://github.com/surik64/Student-Management-System>
